AI4101 · ARTIFICIAL INTELLIGENCE PRINCIPLES · LAB 1

Teaching a Computer to See

Build and test your first image classifier with Teachable Machine

Work	In pairs. One report per pair.
Where	At home, on your own laptop.
Time	About 2 hours.
Due	One week after this lab is posted. Your instructor announces the exact date in class.
Grade	**Not graded.** This lab is practice for how every lab in this course is written and submitted. You will get feedback on your report.
Submit	A short report (PDF), with a link to a video of 3 minutes or less, the way your instructor announces in class.
Links to the course	Module 1 (sections 1.2, 1.4, 1.6) and Module 2 (sections 2.3, 2.5).

What you will learn

- Train an image classifier from your own examples, and test it on objects it has never seen.
- See by experiment that a learned model can only repeat what its **training data** showed it.
- Describe exactly what your model can do, and where it fails.
- Connect the model to the ideas of Modules 1 and 2: agent, PEAS, learning, and responsibility.
- Practise the report format you will use in every lab of this course.

Words you will use

In Module 1 you met **machine learning**: systems that learn patterns from data to improve at a task, rather than relying only on explicitly programmed rules. This lab is your first hands-on look at it. Six more words:

Word	Meaning in this lab
Class	One category the model can choose, for example *Water bottle*. You decide the classes and their names.
Example	One image you give the model for a class. Teachable Machine calls these *image samples*.
Training data	All the examples of all the classes together. The model learns only from these.
Training	The step where the computer looks for patterns that separate your classes. You press **Train Model** and wait.
Model	The result of training: a program that takes a new image and outputs a class.
Confidence	The percentage the model shows for each class. The percentages of all classes add up to 100%. A high percentage does not mean the answer is correct.

Before you start

- **Google Chrome** on a laptop. Teachable Machine works best in Chrome on a computer, not on a phone.
- **A camera.** The laptop webcam is enough. No webcam? Take photos with your phone, send them to the laptop, and use **Upload** instead of **Webcam** in every step below.
- **The objects:** a water bottle and a mobile phone for Part 1, and the objects for the problem you choose in Part 2 (read Part 2 before you start). For Experiment C you also need one object that belongs to none of your classes, such as a key or a pen.
- **A way to take screenshots:** `Windows + Shift + S` on Windows, or `Cmd + Shift + 4` on a Mac.
- **A way to record your screen with your voice:** the Snipping Tool (Record) or Xbox Game Bar (`Windows + Alt + R`) on Windows, or `Cmd + Shift + 5` on a Mac.
- **A YouTube account**, to upload your video as *Unlisted*.
- **The report template** from the lab page `phdahmad.github.io/AI4101/labs/lab1`: `AI4101_M1_Lab1_TeachableMachine_Choice_ReportTemplate.docx`. Open it now and fill it in as you go.

> **Save your work**
>
> Teachable Machine does not save your project on its own. If you close the tab, your examples are gone. To keep a project, open the menu (☰) at the top left and choose **Save project to Drive** or **Download project as file**.

Part 1 · Warm-up: rebuild the class example (15 min)

You saw this in the lecture. Now build it yourself, so you know the tool before the real task.

1. Open teachablemachine.withgoogle.com and click **Get Started**.
2. Choose **Image Project**, then **Standard image model**.
3. Click the pencil next to *Class 1* and rename it **Water bottle**. Rename *Class 2* to **Mobile phone**.
4. Under *Water bottle*, click **Webcam**, then keep the **Hold to Record** button pressed while you slowly turn the bottle. Stop at about **50 examples**. Move the bottle closer and further, and turn it to show different sides.
5. Do the same for *Mobile phone*, with about the same number of examples.
6. Click **Train Model**. Keep the tab open and on the screen until training finishes.
7. In the **Preview** panel, hold each object in front of the camera and watch the percentages.

When both objects are recognised, you are ready. Start a new project for Part 2 (menu ☰ → **New Project**).

Part 2 · Your own classifier: choose a problem (40 min)

Choose **one** problem with your partner. Each problem has three classes and one borderline object that you will use in Part 4. If you cannot decide, take the first one.

Problem	The three classes	Borderline object (Part 4)
Sorting waste for recycling (default)	Plastic bottle · Paper · Metal can	A crushed plastic bottle, and a paper cup (paper outside, a thin plastic layer inside)
Types of dates	Sukkari · Ajwa · Khalas	A dry or broken date
Banana ripeness	Green · Yellow · Brown	A banana that is half green and half yellow
Saudi coins	1 riyal · 2 riyals · 50 halalas	A coin shown on the side you did not record

Or propose your own problem

You may choose a different problem if it meets **all four** conditions:

1. **Three classes of real objects** that you can hold in front of the camera. No pictures downloaded from the internet: the experiments need real objects.
2. **At least two objects per class** for training, plus **five more objects** from your three classes that you keep aside for testing (Part 3 uses them).
3. **One natural borderline object** that you and your partner may disagree about.
4. **No faces or bodies of people.** Pictures of people are personal data (Module 1, section 1.6).

> **Why these conditions?**
>
> Each condition protects one part of the lab. Real objects make Experiment B (the background) possible. Test objects kept aside make the test in step 4 below fair. The borderline object is Part 4. And a system that learns from pictures of people raises privacy questions this lab is not built for.

Build it

1. Click **Add a class** to get a third class. Name the three classes. Write your problem and class names in section 1 of the report.
2. Record **at least 50 examples** for each class. Use more than one object per class (for example, two different bottles). Vary the angle, the distance, and which hand holds the object.
3. Train the model.
4. Now test the model on the objects you kept aside, which are **not** in your training data. Write the results in section 2 of the report.

> **Screenshot 1 — required**
>
> Take a screenshot of the model classifying a **new** object correctly: the object in the camera, and the percentages in the Preview panel. Paste it into section 3 of the report.

Why "new" objects?

Testing the model on the objects you trained it with only tells you that it remembers them. The real question is whether it works on objects it has never seen. That is the test that matters.

Part 3 · Three experiments (40 min)

Each experiment changes one thing and checks what happens. Use the same **5 new test objects** in Experiment A so that the numbers can be compared. Record every result in the table in section 4 of the report.

Experiment A — How much data does the model need?

1. Start a new project with the same three classes, but record only **10 examples** per class. Train it.
2. Test it on your 5 new test objects. Count how many it gets right.
3. Compare with your Part 2 model (50 or more examples per class) on the same 5 objects.

Experiment B — Change the background

1. Go back to your Part 2 model. If all your examples were recorded in front of one background (a wall, a desk), that is your *training background*.
2. Test the same objects in front of a **different** background: a patterned bed cover, a busy shelf, or a bright window.
3. Does the model still get them right? Does the confidence change?

Experiment C — Something that belongs to no class

1. Show the model an object that belongs to none of your three classes: a key, a pen, your hand.
2. Write down the class it chooses and the confidence.
3. Think: why does the model not say "I don't know"?

Screenshot 2 — required

Take a screenshot of a **failure** from any experiment: the model gives a wrong class, or it is unsure. Paste it into section 5 of the report and explain what you think went wrong.

Part 4 · The borderline case (15 min)

Test your Part 2 model on the **borderline object** of your problem: the last column of the table in Part 2, or the one you chose for your own problem. The waste problem has two borderline objects; test both.

Write down what the model says. Then discuss with your partner: **which class is correct?** You may not agree. That is expected.

A question to settle the argument

Two different questions are hiding here. *What should the correct label be?* is a question about the world and the rules of your problem, and the designer must decide it. *What did the model answer, and why?* is a question about the training data: did the model ever see an object like this, and under which label?

Answer both questions in section 6 of the report.

Screenshot 3 — required

Take a screenshot of the model's answer for your borderline object (for the waste problem, one of the two). Paste it into section 6 of the report.

Part 5 · Optional: look inside the model

This part is optional. Skip it if you are short of time.

In the training panel, open **Advanced**, then **Under the hood**. You will find two things:

- **Accuracy per class** — how often the model was right for each class, measured on examples it did not train on. Teachable Machine keeps part of your examples aside to measure this.
- **Confusion matrix** — a table that shows which class was mistaken for which. Each row is the true class; each column is the class the model chose. Numbers off the diagonal are mistakes.

If you use it, add a screenshot and one sentence to section 8 of the report: which two classes does your model confuse most, and why do you think that is?

Part 6 · Connect it to the course

Answer these five questions in section 7 of the report. Use the words from the course site. Short, precise answers are better than long ones.

1. **Agent or not?** In Module 1 (section 1.2), an agent perceives its environment and acts upon that environment. Is the Preview panel on its own an agent? Give one reason for your answer.
2. **PEAS.** Imagine your model inside a system that **acts** on what it sees: a recycling bin that opens the correct compartment, a machine that drops each date into the box of its type, a shop shelf that moves bananas by ripeness, or a machine that counts coins. Write the PEAS for the system that fits your problem (Module 2, section 2.3). Remember: P describes what counts as success, not how the system works.
3. **Designer or data?** Which parts of your model did **you** decide, and which parts did the model **learn** from the examples? (Module 2, section 2.5: a learning agent improves its behaviour from experience.)
4. **Who could it fail?** Look at your results from Experiments B and C. Describe one situation where your system would act wrongly, and who would be affected. (Module 1, section 1.6: bias can come from the data.)
5. **Be precise.** Write one sentence that states exactly what your model can do and under which conditions. Module 1 (section 1.4) gives the style: *"this model recognises five diseases across three plant species from clear images."*

Record your video (3 minutes or less)

One partner records the screen and speaks. You do not need to show your face. The video must show, in this order:

1. The model classifying a **new** object correctly (Screenshot 1, live).
2. **One failure** from your experiments, and your explanation of why it happened.
3. **The borderline case**: the model's answer, and what you and your partner decided.
4. **One sentence**: the most important thing you learned.

Upload it to YouTube:

1. On YouTube, click **Create** → **Upload video**, and choose your file.
2. Give it the title `AI4101 Lab 1 — <your two student IDs>`.
3. On the **Visibility** step, choose **Unlisted**. Not *Public*, not *Private*. Unlisted means only people with the link can watch it.
4. Copy the link. Open it in a private browser window to check that it plays.

Submit

1. Fill in every section of the report template, including the video link on the first page.
2. Save it as PDF with the name `AI4101_Lab1_<ID1>_<ID2>.pdf`. Replace ID1 and ID2 with your two student IDs.
3. **One** partner submits it before the due date, the way your instructor announces in class. Both names and student IDs must be on the first page.

> **Using AI tools**
>
> You may use an AI assistant to help with your English. The screenshots, the results and the ideas must be your own. Section 9 of the report asks you to say which tools you used and for what. This is the rule for every lab in this course.

Checklist before you upload

☐	Both names and student IDs are on the first page.
☐	The video link works in a private window, and the video is Unlisted and 3 minutes or less.
☐	Three required screenshots, each with its explanation.
☐	The experiments table is filled in with numbers, not words like "good".
☐	All five course questions are answered.
☐	The file is a PDF with the correct name.

If something goes wrong

Problem	What to try
The camera does not start.	Click the lock icon next to the address bar and set **Camera** to **Allow**. Close Zoom, Teams or any other app that is using the camera, then reload the page.
My laptop has no webcam.	Take the photos with your phone, send them to the laptop, and use **Upload** instead of **Webcam**.
Training is stuck or very slow.	Keep the Teachable Machine tab visible until training ends. Close other tabs. Use Chrome.
The model gives one class for everything.	Check that every class has about the same number of examples, and that the examples really show different objects.
I lost my project.	Teachable Machine does not save by itself. Rebuild it, and use menu ☰ → **Save project to Drive** this time.
My video is longer than 3 minutes.	Trim it in YouTube Studio (**Content** → your video → **Editor**), or record again with a plan.
My PDF is too big to submit.	Check that it is a PDF. Large screenshots can make the file big; paste them smaller, or use **Compress Pictures** in Word before saving as PDF.

Where this goes next

In this lab the intelligence came from the **data**: you gave examples, and the model found the patterns. Much of this course looks at a different way to be intelligent: agents that find a good decision by **searching**, with no training data at all. Keep your report, so you can look back at the difference.